

Technical Plan: Pitch–Investor Matching & Notification System

Prepared by: Deepak Aryal (deepakaryal2050@gmail.com)

1 Context

The goal is to build a feature that automatically matches investors with newly published pitches and notifies them via email, while ensuring that the pitch publishing process remains fast and unaffected.

To achieve this, the system will adopt an event-driven, queue-based architecture so that matching and notification are handled asynchronously, improving scalability and responsiveness. The implementation will also follow the repository pattern to maintain a clear separation of concerns and ensure the codebase remains maintainable and testable.

2 High Level Architecture

Core components

1. **Pitch service:** It handles the creation and publishing of pitches and dispatches an event after publishing.
2. **Event:** The `PitchWasPublished` event is triggered when a pitch is published.
3. **Listener:** It listens to the dispatched event and handles the matching and notification logic asynchronously.
4. **Queue System:** We can use Laravel queues (Redis, Database, or SQS), which ensure background processing.
5. **Matching Service:** It is a separate service class that encapsulates the investor matching logic.
6. **Notification Service:** It is a separate service class that handles email sending. We can also use this service class to store sent notifications with proper status.

Flow

1. User publishes a pitch.
2. An event is dispatched.
3. The listener is queued.
4. Background worker is triggered and performs the following tasks:
 - Finds matching investors
 - Sends notifications
 - Record results with proper status

3 Matching Logic

A match is successful when all of the following conditions are met:

1. Industry match
2. Investment range match
3. Location match

4 Asynchronous Processing

Problem

Sending emails during the pitch publishing process would slow down the user experience and users should wait longer than expected, as email delivery depends on external services and can introduce latency.

Solution

To avoid this, the matching and notification process should be handled asynchronously using a queue system. When a pitch is published, an event is triggered and pushed to a background queue. A worker then processes this task separately from the main request.

How it works

When a pitch is published, an event is dispatched. A background worker listens for this event and processes it. The worker performs investor matching and sends notifications.

Benefits

Keeps the pitch publishing process fast and responsive Improves system scalability under high load. Allows retries in case of email delivery failures

5 Database Design

industries

- id
- name

pitches

- id
- title
- industry
- funding_amount
- location
- timestamps

Investors

- id
- name
- email

Investor_preferences

- id
- industries (JSON)
- investor_id ([investors.id](#))
- min_investment
- max_investment
- locations (JSON)

Notifications

- id
- investor_id (investors.id)
- pitch_id (pitches.id)
- status
- sent_at
- timestamps

6 Failure Scenarios and Mitigation

1. **Duplicate emails:** Apply unique constraints in the database and also check before sending the notification to ensure that the notification has not already been sent.
2. **Slow queries:** Index the most important attributes such as funding_amount and industry, and filter data directly from the database instead of using PHP.
3. **Email failures:** We can use the retry mechanism provided and log failure details. We should also use the failed_jobs table.
4. **Queue failures:** To keep workers running, we can use Supervisor and also monitor the queues.
5. **Large dataset scaling:** Chunk the data while processing and dispatch smaller jobs per batch if needed.

7 Testing Strategy

Unit tests

Test the matching service logic in detail, including industry matching, investment range boundaries, location edge cases, and more.

Feature tests

Test pitch creation to ensure it triggers the event, the appropriate listener, and that the job executes correctly.

Queue and Notification testing

Assert that the job is dispatched and sent to the correct investors.

Note: I have described the implementation using Laravel concepts (such as events, queues, and jobs), as this is the framework I am most experienced with. However, the same architectural approach can be applied directly in Symfony using its event dispatcher and messenger components.